

Direccionamiento en XPath

Introducción

El **direccionamiento** en XPath es el mecanismo mediante el cual se especifican las rutas para localizar nodos en un documento XML. Existen diferentes formas de expresar estas rutas, cada una con sus propias características y usos.

Tipos de rutas

Rutas absolutas

Una **ruta absoluta** comienza desde el nodo raíz del documento y se identifica porque empieza con una barra diagonal `/`. Especifica la ruta completa desde la raíz hasta los nodos deseados.

Sintaxis:

```
/paso1/paso2/paso3/...
```



Ejemplos:

```
/biblioteca/libro/titulo
```



Selecciona todos los elementos `<titulo>` que sean hijos de `<libro>`, que a su vez son hijos de `<biblioteca>`, que es hijo del nodo raíz.

```
/biblioteca/libro[1]/autor
```



Selecciona el elemento `<autor>` del **primer** libro de la biblioteca.

Ventajas:

- Precisas y predecibles
- No dependen del contexto actual

Desventajas:

- Pueden ser largas si la estructura es profunda
- Menos flexibles ante cambios en la estructura del documento

Rutas relativas

Una **ruta relativa** comienza desde el nodo de contexto actual (no desde la raíz) y no empieza con `/`. Son útiles cuando ya se está posicionado en un nodo específico del documento.

Sintaxis:

```
paso1/paso2/paso3/...
```



Ejemplos:

```
libro/titulo
```



Desde el nodo actual, selecciona todos los elementos `<titulo>` que sean hijos de elementos `<libro>` que a su vez son hijos del nodo actual.

```
./autor
```



Selecciona los elementos `<autor>` hijos del nodo actual (el punto `.` representa el nodo actual).

```
../precio
```



Selecciona los elementos `<precio>` que son **hijos del padre del nodo actual** (los dos puntos `..` representan el nodo padre). Por tanto, selecciona los elementos `<precio>` **hermanos** del nodo actual.

Ventajas:

- Más cortas y concisas
- Útiles en contextos específicos (como dentro de predicados o plantillas XSLT)

Desventajas:

- Dependen del contexto actual

- Pueden ser ambiguas si no se conoce el contexto

Rutas recursivas

Las **rutas recursivas** utilizan la doble barra diagonal `//` para buscar nodos en cualquier nivel de profundidad del árbol, no solo en los hijos directos.

Sintaxis:

```
//nombre-nodo
```



Ejemplos:

```
//titulo
```



Selecciona todos los elementos `<titulo>` en cualquier parte del documento, sin importar su profundidad.

```
/biblioteca//autor
```



Selecciona todos los elementos `<autor>` que sean **descendientes** de `<biblioteca>`, sin importar cuántos niveles hay entre ellos.

```
//libro[@año="2020"]//titulo
```



Selecciona todos los elementos `<titulo>` que sean descendientes de elementos `<libro>` con atributo `año` igual a "2020".

Ventajas:

- Muy flexibles para estructuras variables
- No requieren conocer la estructura completa del documento

Desventajas:

- Pueden ser menos eficientes (buscan en todo el árbol)
- Pueden devolver más resultados de los esperados

Localización

La **localización** se refiere al proceso de identificar nodos específicos dentro de un documento XML. Una expresión de localización consta de uno o más **pasos de localización** (*location steps*) separados por barras diagonales.

Componentes de un paso de localización

Cada paso de localización tiene tres partes:

```
eje::prueba-nodo[predicado]
```



1. **Eje** (*axis*): Define la dirección de búsqueda desde el nodo de contexto
2. **Prueba de nodo** (*node test*): Especifica qué tipo de nodos seleccionar
3. **Predicado** (*predicate*): Filtra los nodos seleccionados (opcional)

Ejemplo completo:

```
child::libro[position() = 1]/child::titulo
```



Desglose:

- `child::` es el eje (selecciona los hijos del nodo actual)
- `libro` es la prueba de nodo (selecciona nodos elemento llamados "libro")
- `[position() = 1]` es el predicado (filtra solo el primer elemento)

Forma abreviada:

La mayoría de expresiones XPath usan formas abreviadas que son equivalentes:

```
libro[1]/titulo
```



Esta es la forma abreviada del ejemplo anterior, donde:

- `child::` se omite (es el eje por defecto)
- `position() = 1` se abrevia como `[1]`

A continuación, se describen las formas de localizar diferentes tipos de nodos en XPath. Para todas ellas, consideremos el siguiente documento XML de ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<europa>
  <pais>
    <!-- Informacion sobre Alemania-->
    <nombre>Alemania</nombre>
    <habitantes unidad="millones">82.4</habitantes>
    <capital>Berlín</capital>
    <sigla-pais>DE</sigla-pais>
    <prefijo>0049</prefijo>
  </pais>
  <pais>
    <!-- Informacion sobre España-->
    <nombre>España</nombre>
    <habitantes unidad="millones">41.2</habitantes>
    <capital>Madrid</capital>
    <sigla-pais>ES</sigla-pais>
    <prefijo>0034</prefijo>
  </pais>
</europa>
```



Localizar el nodo raíz

La ruta de localización del nodo raíz de un documento XML es la siguiente:

/



Localizar elementos

La localización de elementos permite seleccionar todos los hijos del nodo de contexto con el nombre especificado.

Para obtener todos los nodos `<pais>` que contiene el nodo `<europa>`, se utilizaría la siguiente expresión XPath:

/europa/pais



Para obtener todos los nodos `<nombre>` que contienen los nodos `<pais>`, se utilizaría la siguiente expresión XPath:



```
/europa/pais/nombre
```

Localizar descendientes

Para localizar **descendientes**, podemos utilizar dos barras diagonales:

```
//
```



Se trata de una forma abreviada de la siguiente expresión XPath:

```
/descendant-or-self::node()/
```



Por ejemplo, para localizar todos los descendientes `<nombre>` de `<europa>`, se utilizaría la siguiente expresión XPath:

```
/europa//nombre
```



Localizar atributos

Para hacer referencia a un atributo en XPath se emplea el símbolo `@` seguido del nombre del atributo deseado.

Para obtener el atributo millones que contiene el nodo `<habitantes>`, se usaría la siguiente expresión:

```
/europa/pais/habitantes/@unidad
```



Localizar comentarios

Para hacer referencia a los comentarios que están dentro de un nodo se utiliza la función `comment()`.

Para obtener los nodos comentarios que hay dentro del nodo `<pais>`, se utilizaría la siguiente expresión XPath:

```
/europa/pais/comment()
```



Localizar nodos de texto

Para hacer referencia al texto que hay dentro de un nodo, se utiliza la función `text()`.

Para obtener el texto que contiene el nodo `<nombre>` usaríamos la siguiente expresión XPath:

```
/europa/pais/nombre/text()
```



Para obtener el texto que contiene el atributo millones usaríamos la siguiente expresión XPath:

```
/europa/pais/habitantes/@unidad/text()
```



Localizar instrucciones de procesamiento

Para hacer referencia al nodo de instrucciones de procesamiento del elemento se usará la función `processing-instruction()`.

Para obtener las instrucciones de procesamiento que hay dentro del nodo raíz, se utilizaría la siguiente expresión XPath:

```
/processing-instruction()
```



Comodines

Los **comodines** (*wildcards*) permiten seleccionar múltiples tipos de nodos sin especificar nombres exactos. Son útiles cuando se desea flexibilidad en la selección.

Asterisco (*)

El asterisco `*` coincide con cualquier nodo elemento, independientemente de su nombre.

Ejemplos:

```
/biblioteca/*
```



Selecciona todos los elementos hijos de `<biblioteca>`, sin importar su nombre.

```
//libro/*/autor
```



Selecciona todos los elementos `<autor>` que sean nietos de `<libro>` (con cualquier elemento intermedio).

```
//*
```



Selecciona todos los elementos del documento.

Comodín de atributos (@*)

El comodín `@*` selecciona todos los atributos de un elemento.

Ejemplos:

```
//libro/@*
```



Selecciona todos los atributos de todos los elementos `<libro>`.

```
//libro[@*]
```



Selecciona todos los elementos `<libro>` que tengan al menos un atributo.

Función node()

La función `node()` coincide con cualquier tipo de nodo (elemento, texto, comentario, instrucción de procesamiento, etc.).

Ejemplos:

```
//libro/node()
```



Selecciona todos los nodos hijos de `<libro>`, incluyendo elementos, texto y comentarios.

```
/*/node()
```



Selecciona todos los nodos hijos del elemento raíz.

Predicados

Un predicado es una expresión booleana que añade un nivel de verificación al paso de localización. En estas expresiones podemos incorporar funciones XPath. Mientras las rutas de localización seleccionan un conjunto de nodos, los predicados filtran ese conjunto basándose en condiciones específicas.

Los **predicados** se expresan entre corchetes `[]` y filtran el conjunto de nodos seleccionado. Se evalúan para cada nodo en el conjunto de contexto, y solo los nodos para los cuales el predicado es **verdadero** se incluyen en el resultado.

Sintaxis básica

- Utilizan corchetes `[]` para encerrar la condición.
- Pueden contener expresiones numéricas, de comparación, lógicas o funciones. Pueden unirse mediante operadores lógicos como `and`, `or`, y `not()`.
- Dentro de un predicado podemos incluir **ejes**.

Predicados numéricos

Filtran por posición en el conjunto de nodos.

Ejemplos:

```
//libro[1]
```



Selecciona el primer elemento `<libro>` de cada contexto.

```
//libro[last()]
```



Selecciona el último elemento `<libro>` de cada contexto.

```
//libro[position() < 4]
```



Selecciona los tres primeros elementos `<libro>`.

```
//libro[position() mod 2 = 0]
```



Selecciona los elementos `<libro>` en posiciones pares.

Predicados de comparación

Comparan valores de nodos o atributos.

Ejemplos:

```
//libro[precio < 20]
```



Selecciona los libros cuyo precio es menor que 20.

```
//libro[@idioma = 'español']
```



Selecciona los libros con atributo `idioma` igual a 'español'.

```
//libro[año >= 2020]
```



Selecciona los libros publicados en 2020 o después.

```
//libro[título = 'Don Quijote']
```



Selecciona los libros cuyo título es exactamente 'Don Quijote'.

Predicados lógicos

Combinan múltiples condiciones con operadores lógicos.

Ejemplos:

```
//libro[precio < 20 and @stock > 0]
```



Selecciona libros con precio menor a 20 Y con stock disponible.

```
//libro[año = 2020 or año = 2021]
```



Selecciona libros publicados en 2020 O en 2021.

```
//libro[not(precio > 30)]
```



Selecciona libros cuyo precio NO es mayor que 30 (es decir, menor o igual a 30).

Predicados con funciones

Utilizan funciones XPath para filtrar nodos.

Ejemplos:

```
//libro[contains(titulo, 'Quijote')]
```



Selecciona libros cuyo título contiene la palabra 'Quijote'.

```
//libro[starts-with(autor, 'Miguel')]
```



Selecciona libros cuyo autor comienza con 'Miguel'.

```
//libro[string-length(titulo) > 20]
```



Selecciona libros cuyo título tiene más de 20 caracteres.

```
//autor[count(libro) > 5]
```



Selecciona autores que tienen más de 5 libros.

Múltiples predicados

Se pueden encadenar varios predicados, evaluándose de izquierda a derecha.

Ejemplos:

```
//libro[precio < 30][año > 2015]
```



Selecciona libros con precio menor a 30 que además fueron publicados después de 2015.

```
//libro[@idioma='español'][1]
```



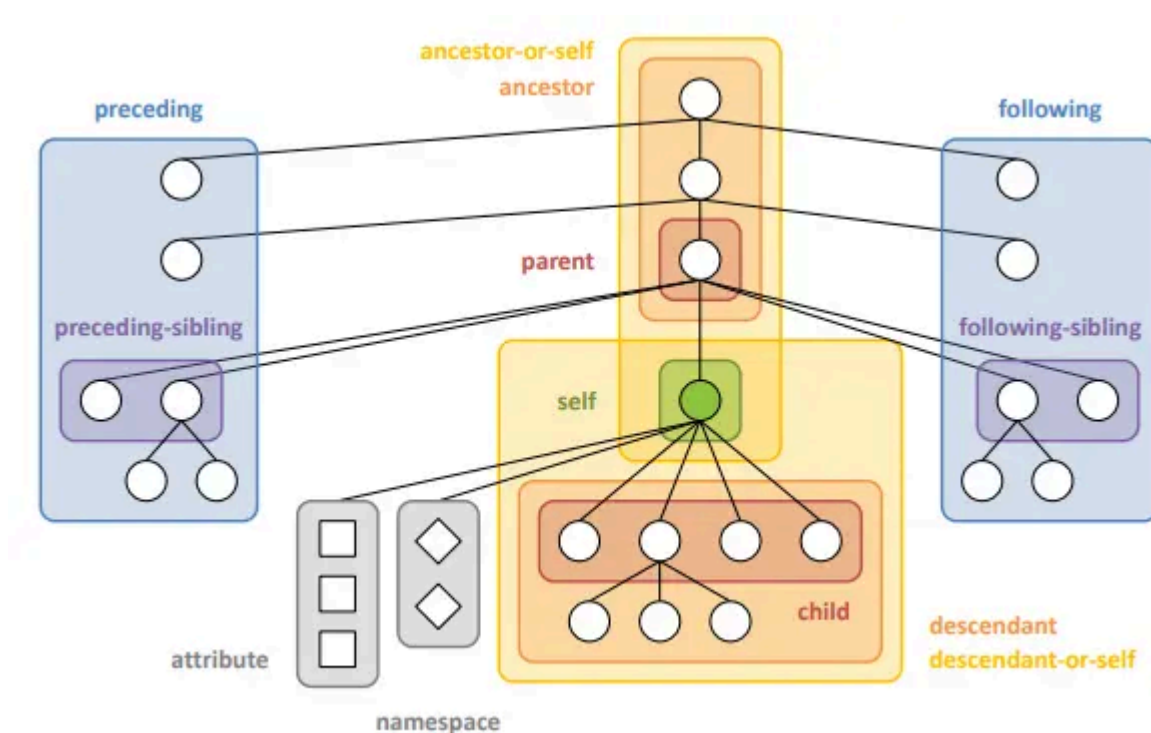
Selecciona el primer libro en español (el primer libro que cumple la condición del primer predicado).

⚠ ORDEN DE EVALUACIÓN

El orden de los predicados importa. `//libro[@idioma='español'][1]` selecciona el primer libro en español, mientras que `//libro[1][@idioma='español']` selecciona el primer libro solo si es en español.

Ejes

Los **ejes** (*axes*) definen la dirección de navegación desde el nodo de contexto y determinan qué nodos se seleccionan en relación con él. Cada eje especifica una relación particular entre el nodo de contexto y los nodos que selecciona. Por lo tanto, nos permiten seleccionar el subárbol dentro del nodo contexto que cumple un patrón determinado.



Sintaxis de ejes

```
eje::prueba-nodo[predicado]
```



Aunque la sintaxis completa usa el doble dos puntos `::`, la mayoría de expresiones usan formas abreviadas.

Ejes principales

child (hijo)

Selecciona todos los **hijos** del nodo de contexto.

Sintaxis completa:

```
child::libro
```



Aunque la forma abreviada es más común:

Forma abreviada:

```
libro
```



Ejemplo:

```
/biblioteca/child::libro
```



Equivale a:

```
/biblioteca/libro
```



descendant (descendiente)

Selecciona todos los **descendientes** del nodo de contexto (hijos, nietos, bisnietos, etc.).

Sintaxis completa:

```
descendant::titulo
```



Forma abreviada:

```
./titulo
```



Ejemplo:

```
/biblioteca/descendant::titulo
```



Selecciona todos los elementos `<titulo>` que sean descendientes de `<biblioteca>`. En su forma abreviada:

```
/biblioteca//titulo
```



parent (padre)

Selecciona el nodo **padre** del nodo de contexto.

Sintaxis completa:

```
parent::libro
```



Forma abreviada:

```
..
```



Ejemplo:

```
//titulo/parent::libro
```



Selecciona todos los elementos `<libro>` que son padres de elementos `<titulo>`. En su forma abreviada:

```
//titulo/..
```



ancestor (ancestro)

Selecciona todos los **ancestros** del nodo de contexto (padre, abuelo, bisabuelo, etc.), incluyendo el nodo raíz.

Sintaxis completa:

```
ancestor::seccion
```



Ejemplo:

```
//titulo/ancestor::libro
```



Selecciona todos los elementos `<libro>` que son ancestros de algún elemento `<titulo>`.

following-sibling (hermano siguiente) #

Selecciona **todos los hermanos** del nodo de contexto que aparecen **después** de él en el documento.

Sintaxis completa:

```
following-sibling::libro
```



Ejemplo:

```
//libro[1]/following-sibling::libro
```



Selecciona todos los libros que aparecen después del primer libro.

preceding-sibling (hermano precedente) #

Selecciona **todos los hermanos** del nodo de contexto que aparecen **antes** que él en el documento.

Sintaxis completa:

```
preceding-sibling::libro
```



Ejemplo:

```
//libro[5]/preceding-sibling::libro
```



Selecciona todos los libros que aparecen antes del quinto libro.

attribute (atributo) #

Selecciona todos los **atributos** del nodo de contexto.

Sintaxis completa:

```
attribute::isbn
```



Forma abreviada:

```
@isbn
```



Ejemplo:

```
//libro/attribute::precio
```



Equivale a:

```
//libro/@precio
```



self (mismo) #

Selecciona el **nodo de contexto** mismo.

Sintaxis completa:

```
self::libro
```



Forma abreviada:

```
.
```



Ejemplo:

```
//libro[self::*[@precio < 20]]
```



Selecciona libros que tienen un atributo `precio` menor que 20. En su forma abreviada:

```
//libro[.[@precio < 20]]
```



descendant-or-self (descendiente o mismo)

Selecciona el nodo de contexto y todos sus **descendientes**.

Sintaxis completa:

```
descendant-or-self::node()
```



Forma abreviada:

```
//
```



Ejemplo:

```
/biblioteca/ancestor-or-self::libro
```



Selecciona el elemento `<libro>` si `biblioteca` es un libro, más todos los elementos `<libro>` descendientes.

ancestor-or-self (ancestro o mismo)

Selecciona el nodo de contexto y todos sus **ancestros**.

Sintaxis completa:

```
ancestor-or-self::biblioteca
```



Ejemplo:

```
//titulo/ancestor-or-self::*
```



Selecciona el elemento `<titulo>` y todos sus ancestros.

following (siguiente)

Selecciona todos los nodos que aparecen **después** del nodo de contexto en el documento, excluyendo descendientes.

Sintaxis completa:

```
following::libro
```



Ejemplo:

```
//libro[1]/following::libro
```



Selecciona todos los elementos `<libro>` que aparecen después del primer libro en orden del documento.

preceding (precedente)

Selecciona todos los nodos que aparecen **antes** del nodo de contexto en el documento, excluyendo ancestros.

Sintaxis completa:

```
preceding::libro
```



Ejemplo:

```
//libro[5]/preceding::libro
```



Selecciona todos los elementos `<libro>` que aparecen antes del quinto libro en orden del documento.

Tabla resumen de ejes

Eje	Descripción	Forma abreviada
<code>child::</code>	Hijos del nodo de contexto	(ninguna)
<code>descendant::</code>	Descendientes del nodo de contexto	<code>//</code>
<code>parent::</code>	Padre del nodo de contexto	<code>..</code>
<code>ancestor::</code>	Ancestros del nodo de contexto	-
<code>following-sibling::</code>	Hermanos siguientes	-
<code>preceding-sibling::</code>	Hermanos precedentes	-
<code>attribute::</code>	Atributos del nodo de contexto	<code>@</code>
<code>self::</code>	El propio nodo de contexto	<code>.</code>
<code>descendant-or-self::</code>	El nodo de contexto y sus descendientes	<code>//</code>
<code>ancestor-or-self::</code>	El nodo de contexto y sus ancestros	-
<code>following::</code>	Todos los nodos siguientes (no descendientes)	-
<code>preceding::</code>	Todos los nodos precedentes (no ancestros)	-

Ejemplos prácticos con ejes

Ejemplo 1: Navegación compleja

```
<biblioteca>
  <seccion nombre="Narrativa">
    <libro isbn="001">
      <titulo>El Quijote</titulo>
      <autor>Cervantes</autor>
    </libro>
    <libro isbn="002">
      <titulo>La Regenta</titulo>
      <autor>Clarín</autor>
```



```
</libro>
</seccion>
</biblioteca>
```

Expresiones:

```
//libro[@isbn='001']/following-sibling::libro/titulo
```



Resultado: `<titulo>La Regenta</titulo>`

```
//autor[text()='Cervantes']/parent::libro/titulo
```



Resultado: `<titulo>El Quijote</titulo>`

```
//titulo/ancestor::seccion/@nombre
```



Resultado: `nombre="Narrativa"` (para ambos títulos)

Ejemplo 2: Búsqueda de contexto

```
//libro[descendant::autor='Cervantes']
```



Selecciona todos los libros que tienen un descendiente `<autor>` con el texto 'Cervantes'.

```
//autor[ancestor::seccion[@nombre='Narrativa']]
```



Selecciona todos los autores que están dentro de una sección llamada 'Narrativa'.



CUÁNDO USAR CADA EJE

- Usa `child::` (o su forma abreviada) para navegación directa
- Usa `descendant::` cuando no conozcas la profundidad exacta
- Usa `following-sibling::` y `preceding-sibling::` para comparar elementos al mismo nivel
- Usa `ancestor::` para subir en la jerarquía y acceder a contexto superior

- Usa `attribute::` (o `@`) siempre que necesites acceder a atributos